

Relatório de Laboratório

Lab01 — Características de repositórios populares no GitHub

Campo	Informação
Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / período	Noite / 6º
Professor	Danilo Maia
Grupo	Pedro Luis · Enrico Bessa · Pércles Pires
Repositório / Project	https://github.com/PedroL10/Lab01Exp · Project LAB01
Data de entrega	27/08/2026

1. Introdução

Repositórios populares no GitHub são frequentemente tratados como referências de qualidade e boas práticas, mas popularidade não descreve, por si só, maturidade, atividade ou capacidade de manutenção. Este estudo investiga empiricamente essas características em uma amostra dos 1.000 repositórios públicos com mais estrelas, obtida pela API GraphQL oficial do GitHub.

As perguntas de pesquisa e hipóteses informais do grupo são:

- RQ01 — Sistemas populares são maduros/antigos? Hipótese: a maior parte acumulou visibilidade ao longo de vários anos.
- RQ02 — Sistemas populares recebem muita contribuição externa? Hipótese: a visibilidade atrai Pull Requests aceitas.
- RQ03 — Sistemas populares lançam releases com frequência? Hipótese: projetos de destaque mantêm ciclos formais de release.
- RQ04 — Sistemas populares são atualizados com frequência? Hipótese: comunidades ativas produzem pushes recentes.
- RQ05 — Sistemas populares são escritos nas linguagens mais populares? Hipótese: o topo da amostra se aproxima do GitHub Octoverse.
- RQ06 — Sistemas populares possuem alto percentual de issues fechadas? Hipótese: a manutenção ativa eleva a taxa de fechamento.
- RQ07 — Linguagens populares recebem mais contribuição, lançam mais releases e são atualizadas mais frequentemente? Hipótese: a relação existe, mas pode variar entre ecossistemas.
- RQ08 (inovação) — Repositórios sem linguagem primária detectada têm indicadores de manutenção piores? Hipótese: esse grupo contém mais documentação e conteúdo curado do que software em evolução contínua.

2. Contexto

Este é o primeiro laboratório da disciplina e não depende de dados de laboratórios anteriores. O objeto de estudo são repositórios públicos retornados pela busca GraphQL ``stars:>1 sort:stars-desc``, limitada pela própria plataforma aos 1.000 primeiros resultados. A coleta inclui data de criação, último push, linguagem primária, releases, issues e Pull Requests aceitas.

3. Metodologia

3.1 Principais desafios

- Custo e disponibilidade da API: consultas com 50–100 repositórios e conexões aninhadas excederam o custo aceitável e retornaram falhas 502/503/504. A coleta adotou lotes de 10 e retry com backoff exponencial.
- Limite de busca: a API do GitHub retorna no máximo 1.000 resultados por consulta ordenada, mesmo com paginação.
- Ambiente de rede: a rede da instituição exigiu `pip-system-certs` para compatibilidade de certificados SSL no Windows.
- Rastreabilidade de processo: o GitHub Projects não disponibiliza histórico de transições de Status por API; por isso, cada sprint produz um snapshot CSV do quadro.

3.2 Decisões metodológicas

- Python foi escolhido pela disponibilidade de bibliotecas de análise e visualização, sem usar bibliotecas específicas para consulta à API do GitHub.
- PRs aceitas são medidas por `pullRequests(states: MERGED) { totalCount }`, diretamente pela GraphQL, em vez de depender do índice eventualmente consistente da busca REST.
- A análise fixa a referência temporal em 2026-08-24T00:00:00Z. Isso torna reproduzíveis idade do repositório e dias desde o último push.
- Medianas são priorizadas, pois PRs, releases e tempo sem atualização possuem distribuições assimétricas e outliers relevantes.
- A RQ07 exige apenas linguagens com pelo menos 10 repositórios, evitando interpretações a partir de grupos muito pequenos; o tamanho de cada grupo é mostrado nos gráficos.
- O limite de WIP em Doing é 3: no máximo um cartão ativo por integrante do trio.

3.3 Etapas e configuração do processo

Sprint	Entregas	Responsável / Issues
Lab01S01	Autenticação, query de 100 repositórios e validação de dados	Enrico: #1–#3; Pedro: #4–#7
Lab01S02	Paginação para 1.000, CSV, hipóteses e primeira análise	Enrico: #8–#10, #13–#14; Pedro: #11–#12
Lab01S03	Análise e visualizações RQ01–RQ07; inovação RQ08; 15 snapshots de repositório.	Enrico: #15–#17

O Project LAB01 usa o fluxo Backlog → To Do → Doing → Review → Done. Ao final desta sprint, o snapshot `data/snapshot\_lab01\_s03.csv` registra os itens e seus Status. O quadro está disponível em <https://github.com/users/PedroL10/projects/1>.

3.4 Ferramentas

- Python 3.11; requests; python-dotenv; pip-system-certs; Matplotlib.
- API GraphQL oficial do GitHub e GitHub Projects (v2).
- Scripts próprios do grupo: `collect\_repositories.py`, `analyze\_metrics.py`, `visualize\_metrics.py` e `export\_project\_snapshot.py`.

3.5 Tabela de métricas

RQ	Métrica	Definição operacional	Unidade	Fonte
RQ01	Idade	referência – createdAt	dias / anos	GraphQL + análise
RQ02	PRs aceitas	pullRequests(states: MERGED) { totalCount }	contagem	GraphQL
RQ03	Releases	releases.totalCount	contagem	GraphQL
RQ04	Atualização	referência – pushedAt	dias	GraphQL + análise

RQ	Métrica	Definição operacional	Unidade	Fonte
RQ05	Linguagem	primaryLanguage.name vs. category	Categoria	GraphQL + Octoverse
RQ06	Issues fechadas	$100 \times \text{closedIssues} / \text{issues}$	% por repositório	GraphQL + análise
RQ07	Indicadores por linguagem	medianas RQ02/RQ03/RQ06 por primaryLanguage	Comparativa	análise
RQ08	Com/sem linguagem	medianas de manutenção comparativa de primaryLanguage	Comparativa	análise

3.6 Inovação proposta pelo grupo

A RQ08 compara repositórios cujo campo `primaryLanguage` é nulo com aqueles que têm linguagem primária informada. A métrica não pressupõe ausência de código: ela identifica apenas ausência de uma linguagem dominante detectada pela GitHub GraphQL. A comparação aprofunda as RQ05 e RQ07 ao separar repositórios de software tradicional de artefatos populares de documentação, listas e conteúdo curado.

4. Resultados

4.1 Qualidade e cobertura dos dados

A base contém 1000 repositórios sem duplicatas. Os campos `createdAt`, `pushedAt`, contagens de PRs, releases e issues estão preenchidos em todos os registros; 87 repositórios (8,7%) não informam linguagem primária. Para RQ06, 43 repositórios sem issues são excluídos da divisão por zero. Não há casos com issues fechadas acima do total.

A referência temporal da análise é 2026-08-24T00:00:00Z. Os resultados devem ser interpretados como um snapshot: valores de estrelas, pushes, issues e PRs mudam continuamente no GitHub.

4.2 Visualizações e respostas às RQs

RQ01 — Sistemas populares são maduros/antigos? A idade mediana é de 7.8 anos (2829 dias), com intervalo entre 10 e 6708 dias. A resposta é majoritariamente sim, embora a presença de projetos recentes mostre que maturidade não é condição necessária para alcançar grande visibilidade.

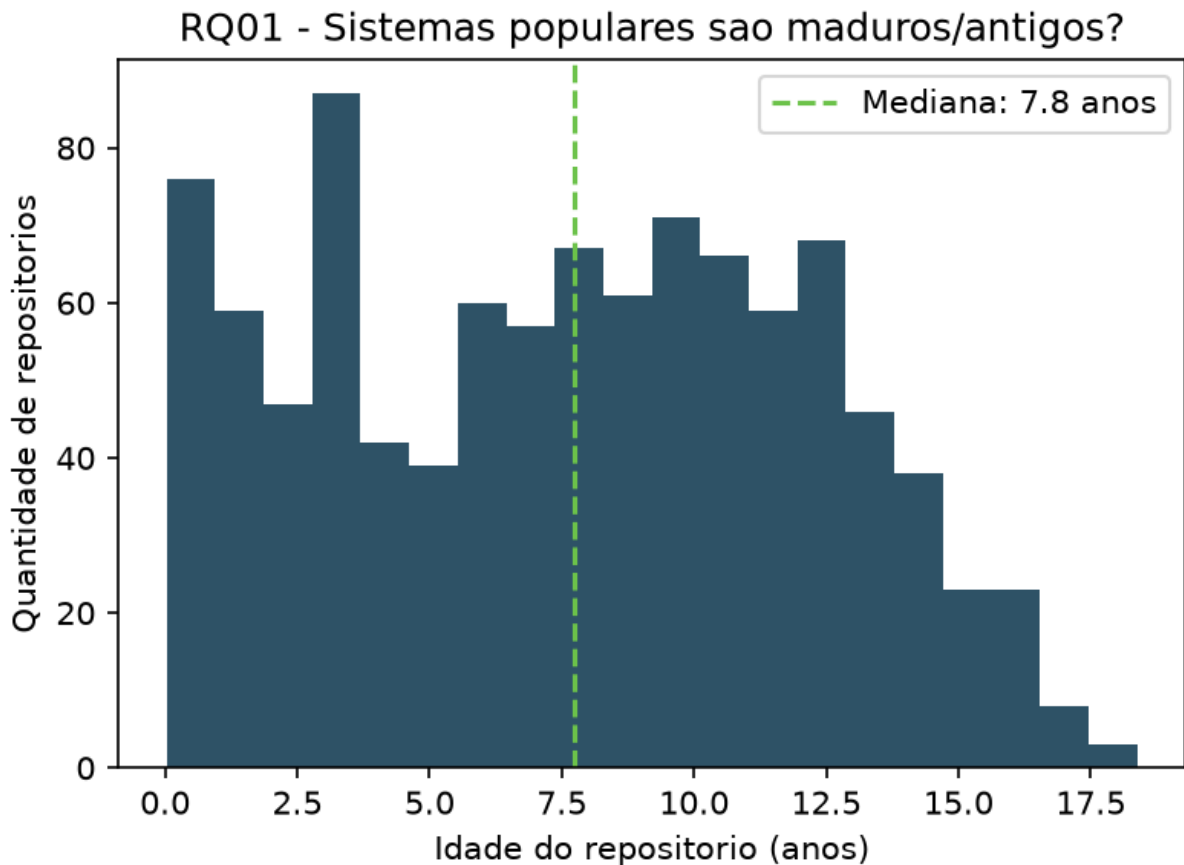


Figura 1 — Distribuição da idade dos repositórios (RQ01).

RQ02 — Sistemas populares recebem muita contribuição externa? A mediana é 768 PRs aceitas e a média é 4243, muito maior devido a outliers. A resposta é sim para a maioria da amostra, com a ressalva de repositórios populares cujo desenvolvimento acontece fora do fluxo de PRs do GitHub.

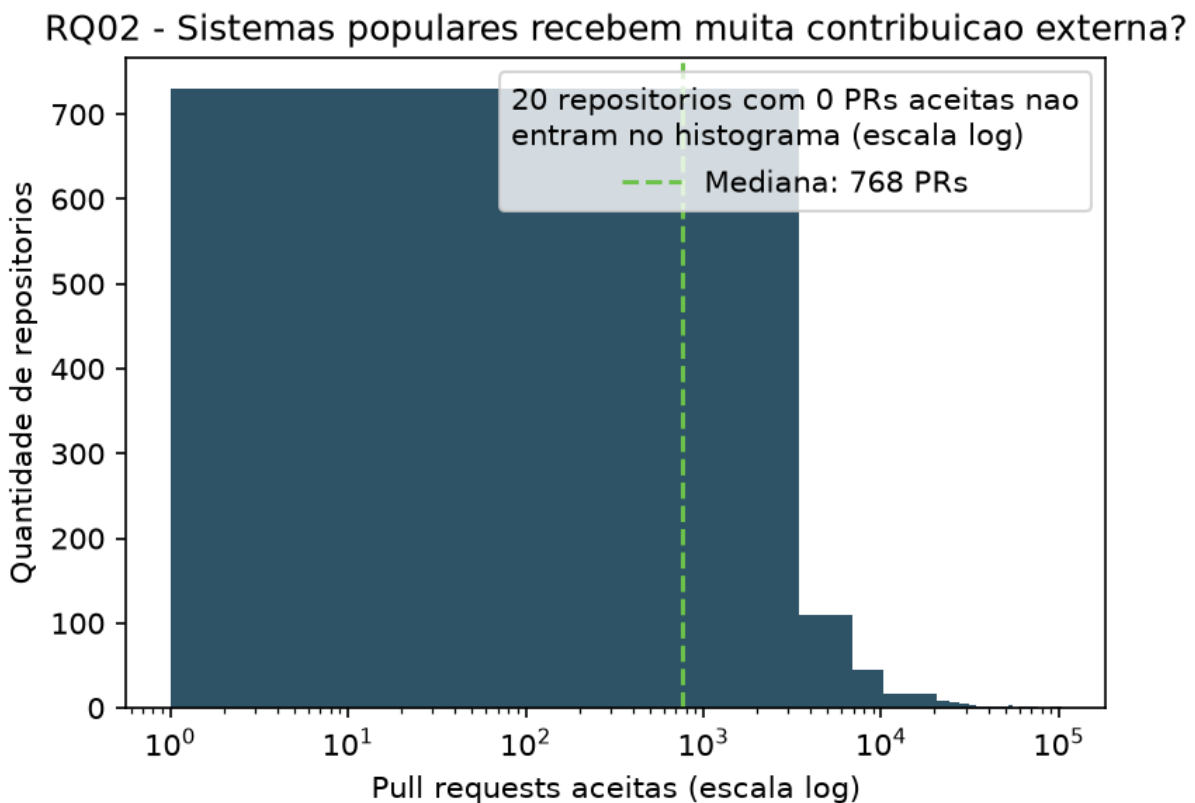


Figura 2 — Distribuição de Pull Requests aceitas (RQ02).

RQ03 — Sistemas populares lançam releases com frequência? A mediana é 39 releases, mas 28.5% não publicaram nenhuma release. A hipótese é parcialmente confirmada: releases formais são comuns, porém não universais entre projetos populares.

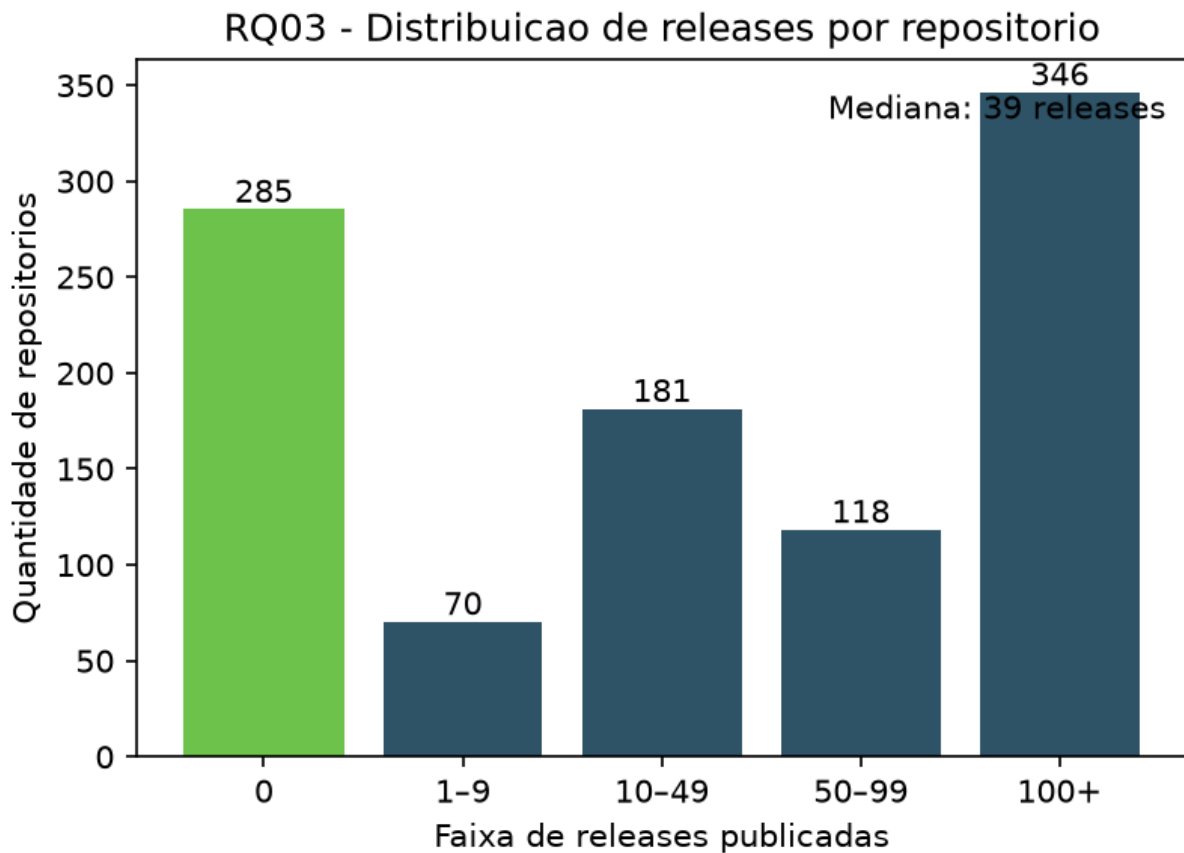


Figura 3 — Distribuição por faixa de releases (RQ03).

RQ04 — Sistemas populares são atualizados com frequência? A mediana de 4 dias desde o último push indica atividade recente para a maior parte dos projetos. A média de 116.66 dias é maior pela presença de repositórios pouco atualizados.

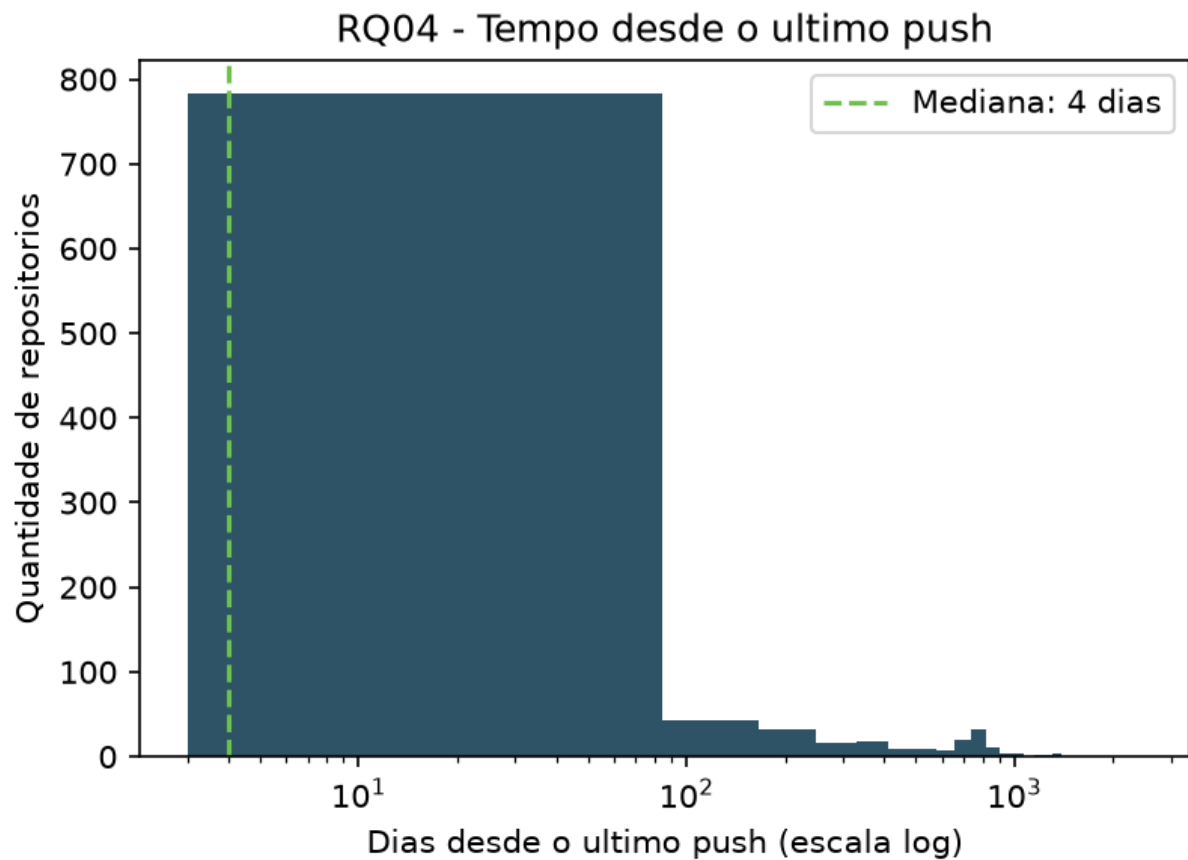


Figura 4 — Distribuição do tempo desde o último push (RQ04).

RQ05 — Sistemas populares são escritos nas linguagens mais populares? Python lidera a amostra com 22,7%, seguido por TypeScript (17,3%) e JavaScript (11,1%). As linguagens mais frequentes são Python (22.7%), TypeScript (17.3%), JavaScript (11.1%), Nao informado (8.7%), Go (7.7%), Rust (5.8%). A liderança de Python é consistente com o GitHub Octoverse 2024; TypeScript supera JavaScript neste recorte de alta visibilidade.

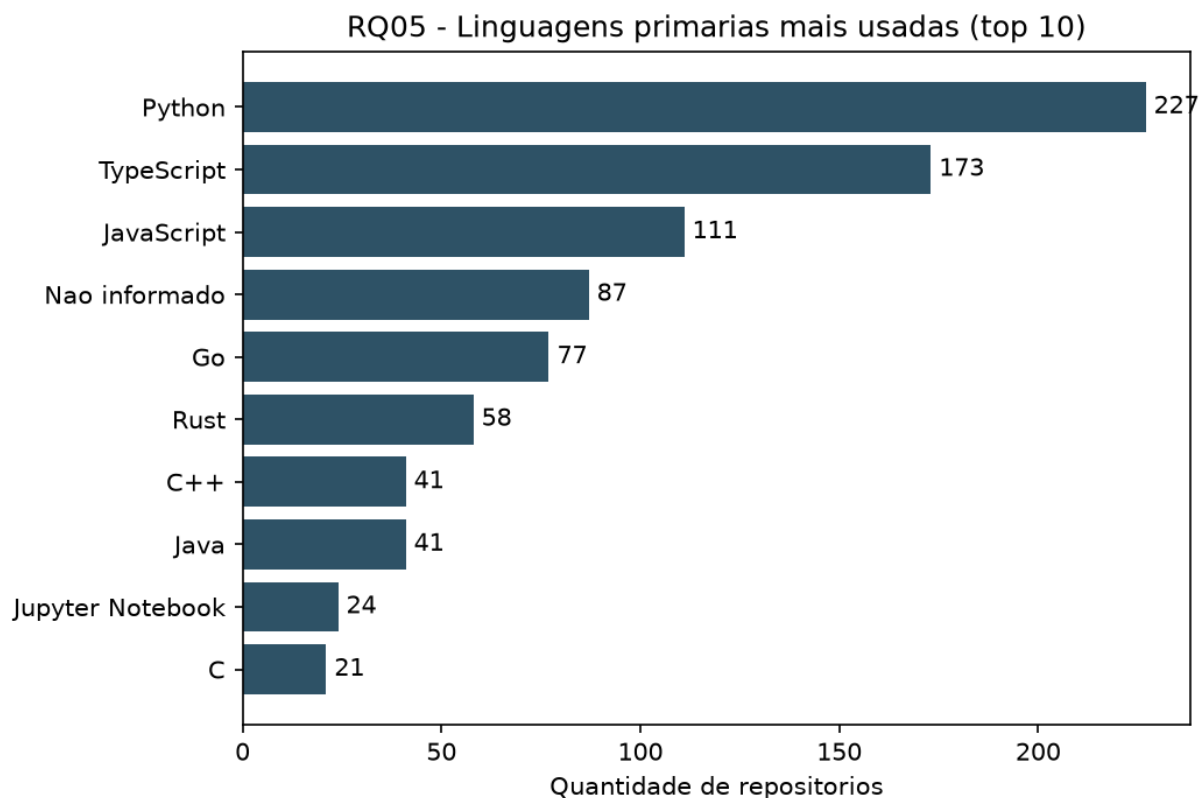


Figura 5 — Linguagens primárias mais frequentes (RQ05).

RQ06 — Sistemas populares possuem alto percentual de issues fechadas? A mediana é 87.5% e a média é 80.2%. A hipótese é confirmada para os repositórios com ao menos uma issue registrada.

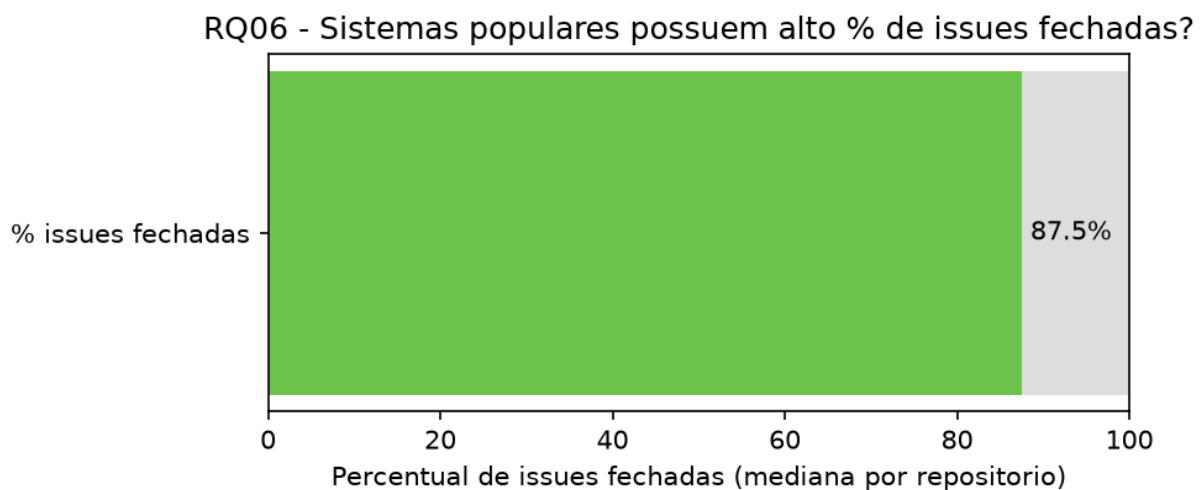


Figura 6 — Percentual mediano de issues fechadas (RQ06).

RQ07 — Linguagens populares recebem mais contribuição, lançam mais releases e são atualizadas mais frequentemente? A resposta é parcialmente confirmada. TypeScript (medianas de 1979 PRs e 133 releases) e Go (1958 PRs e 142 releases) se destacam. Python é a linguagem mais comum, mas tem medianas menores (560 PRs e 20 releases). A relação, portanto, não é linear.

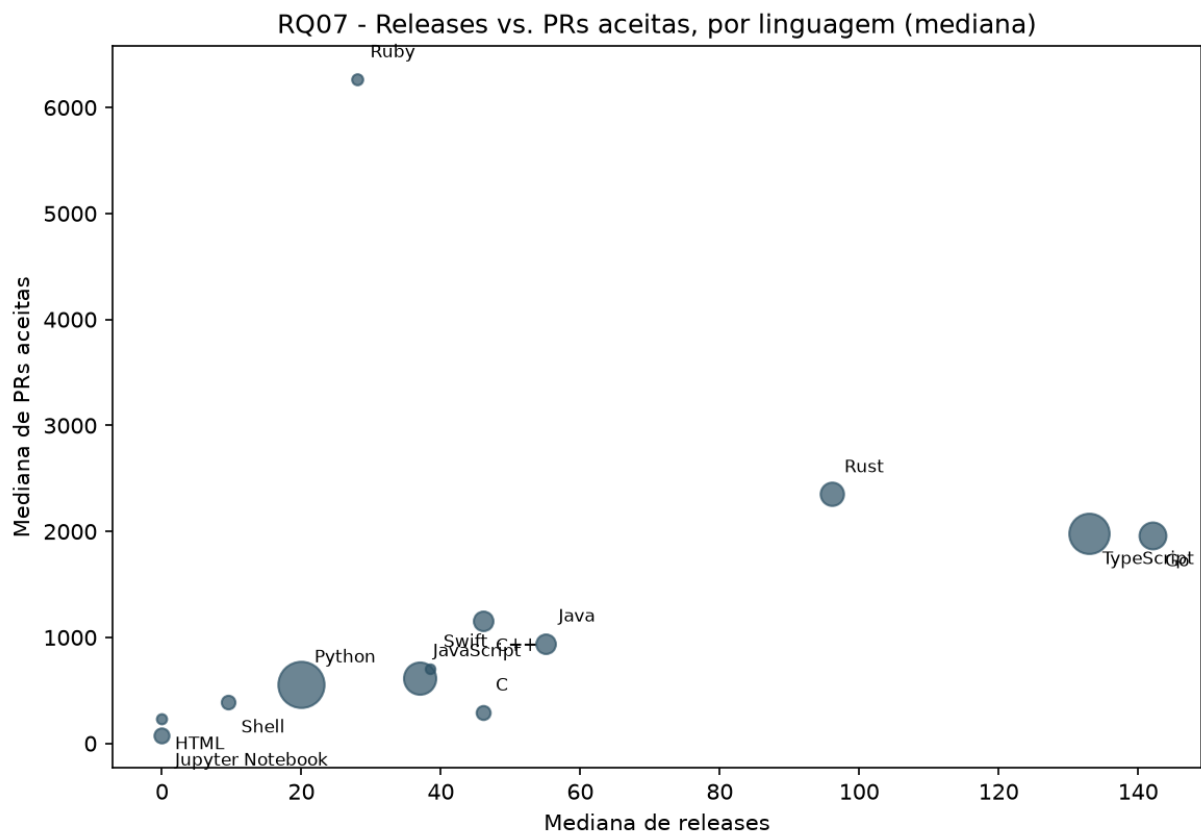


Figura 7 — Releases e PRs aceitas por linguagem (RQ07).

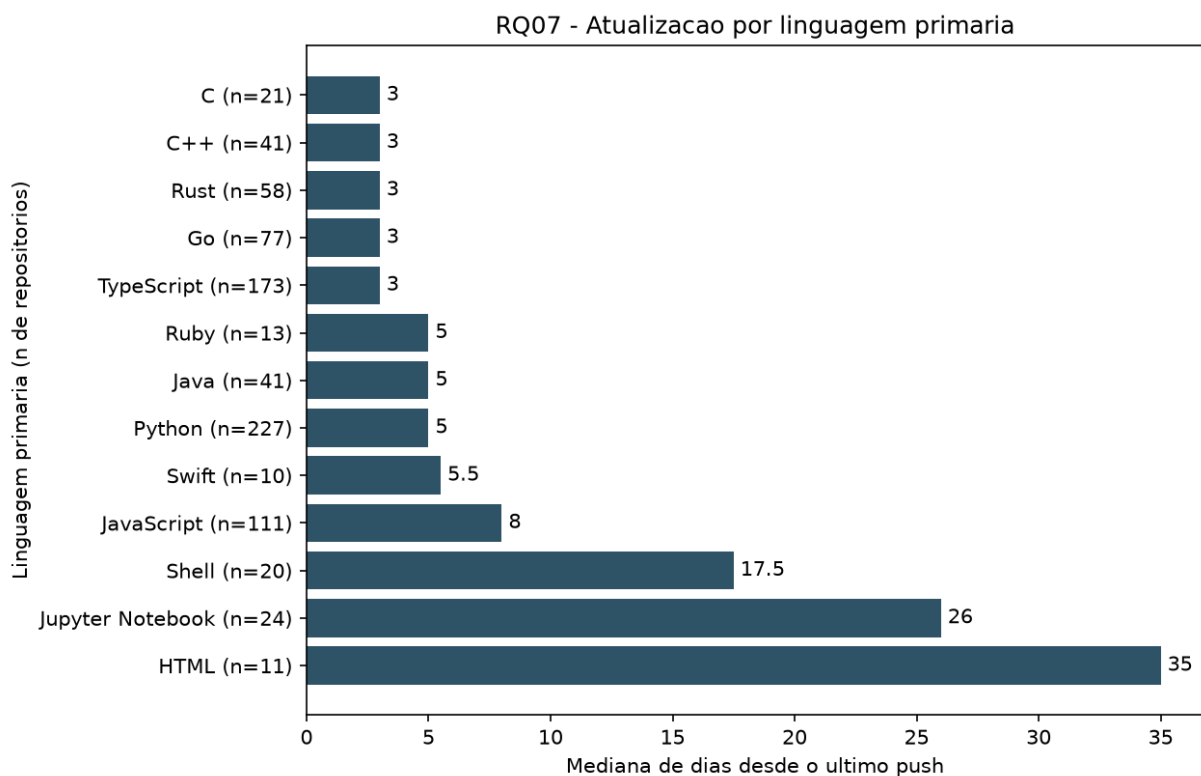


Figura 8 — Atualização por linguagem, grupos com $n \geq 10$ (RQ07).

RQ08 — Repositórios sem linguagem primária detectada têm indicadores de manutenção piores? Sim, de forma exploratória. O grupo sem linguagem contém 87 repositórios, com mediana de 129 PRs, 0 releases e 181 dias sem push; no grupo com linguagem ($n=913$), os valores são 929, 51 e 4, respectivamente.

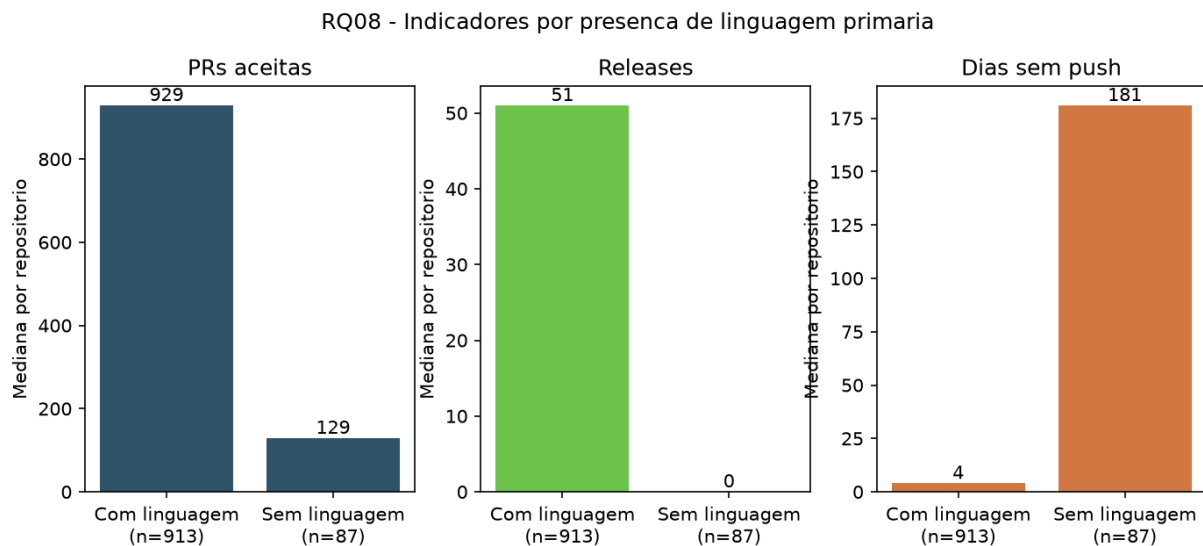


Figura 9 — Indicadores de manutenção por presença de linguagem primária (RQ08).

4.3 Discussão e ameaças à validade

- RQ01 foi parcialmente confirmada: a mediana de 7,8 anos mostra predominância de maturidade, mas projetos recentes também podem alcançar popularidade rapidamente.
- RQ02 foi confirmada na tendência central; no entanto, a contribuição externa pelo GitHub não representa todo o desenvolvimento de projetos espelhados ou mantidos em outras plataformas.
- RQ03 foi parcialmente confirmada: o uso de releases é frequente, mas listas, documentação e materiais educacionais populares não necessariamente adotam versionamento por releases.
- RQ04 foi confirmada: a mediana de quatro dias aponta manutenção recente, enquanto a média maior evidencia uma cauda de projetos pouco atualizados.
- RQ05 foi confirmada para o topo da distribuição. A inversão entre TypeScript e JavaScript sugere que o recorte de projetos muito populares difere da população geral do Octoverse.
- RQ06 foi confirmada: a alta mediana de issues fechadas é compatível com triagem e manutenção ativas, mas não mede a qualidade ou o tempo de resolução de cada issue.
- RQ07 foi parcialmente confirmada: algumas linguagens populares apresentam indicadores altos, porém Python e Ruby mostram que a popularidade da linguagem não prevê, sozinha, contribuição, releases e atualização por repositório.
- RQ08 aprofunda a interpretação: a ausência de linguagem primária está associada a manutenção menos frequente, mas o resultado não prova causalidade. O grupo pode concentrar artefatos de documentação e conteúdo curado, diferentes de software executável.

As principais ameaças à validade são: coleta em um único instante; limite de 1.000 resultados da busca; mudança contínua dos indicadores do GitHub; e comparação com o Octoverse, que cobre todos os repositórios públicos, não apenas os mais estrelados. Além disso, `primaryLanguage` representa bytes de código, não necessariamente a tecnologia mais relevante para usuários do projeto.

5. Conclusão

A amostra indica que repositórios muito populares tendem a ser maduros, receber muitas PRs, manter alta taxa de issues fechadas e sofrer atualizações recentes. Essas características, contudo, não são universais: quase um terço não usa releases formais, e parte dos projetos populares é documentação, lista curada ou mirror, com dinâmica de manutenção distinta.

A segmentação por linguagem mostra que não há relação simples entre popularidade da linguagem e intensidade de contribuição por projeto. TypeScript e Go se destacam nos indicadores de PRs e releases,

enquanto Python lidera em frequência de repositórios. A RQ08 reforça a necessidade de distinguir popularidade de atividade contínua de desenvolvimento.

Com mais tempo, o grupo repetiria a coleta em múltiplas datas e incorporaria análises de tendência e de tempo de resolução de issues. Isso reduziria a dependência de um único snapshot e permitiria testar se os padrões identificados se mantêm ao longo do tempo.

Referências

- GITHUB. GraphQL API documentation. Disponível em: <https://docs.github.com/graphql>. Acesso em: 27 ago. 2026.
- GITHUB. Octoverse 2024: the state of open source. GitHub, 2024. Disponível em: <https://octoverse.github.com/>. Acesso em: 27 ago. 2026.
- GITHUB. About Projects. Disponível em: <https://docs.github.com/issues/planning-and-tracking-with-projects>. Acesso em: 27 ago. 2026.
- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.